

ScikitLearn.jl

ScikitLearn.jl is the Julia version of the popular Scikit-learn library. As you may be aware, ScikitLearn is a very popular option for building ML solutions.

The ScikitLearn.jl is a simple-to-use ML library in Julia with lots of features (<https://scikitlearnjl.readthedocs.io/en/latest/>). The major features of ScikitLearn.jl are listed below:

- Support for DataFrames
- Hyper parameter tuning
- Feature unions and pipelines
- Cross-validation

The availability of a uniform interface to Julia and Python models makes using ScikitLearn.jl effective and simple.

As stated in the beginning of this article, Julia is a powerful language when it comes to handling mathematical expressions. This core advantage is reflected in all the ML libraries built using Julia. In the near future, Julia could emerge as the leading choice for implementing machine

learning solutions for a majority of developers. **END** 

References

- [1] Flux: <https://fluxml.ai/Flux.jl/stable/>
- [2] MetalHead: <https://github.com/FluxML/Metalhead.jl>
- [3] Knet.jl: <http://denizyuret.github.io/Knet.jl/latest/index.html>
- [4] MLBase.jl: <https://mlbasejl.readthedocs.io/en/latest/index.html>
- [5] Julia Observer: <https://juliaobserver.com/categories/Machine%20Learning>
- [6] TensorFlow.jl: https://github.com/malmaud/TensorFlow.jl/blob/master/docs/src/why_julia.md
- [7] Scikitlearn.jl: <https://github.com/cstjean/ScikitLearn.jl>

 By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry. He can be reached via mail at kskuppup@gmail.com.

Continued From Page...88

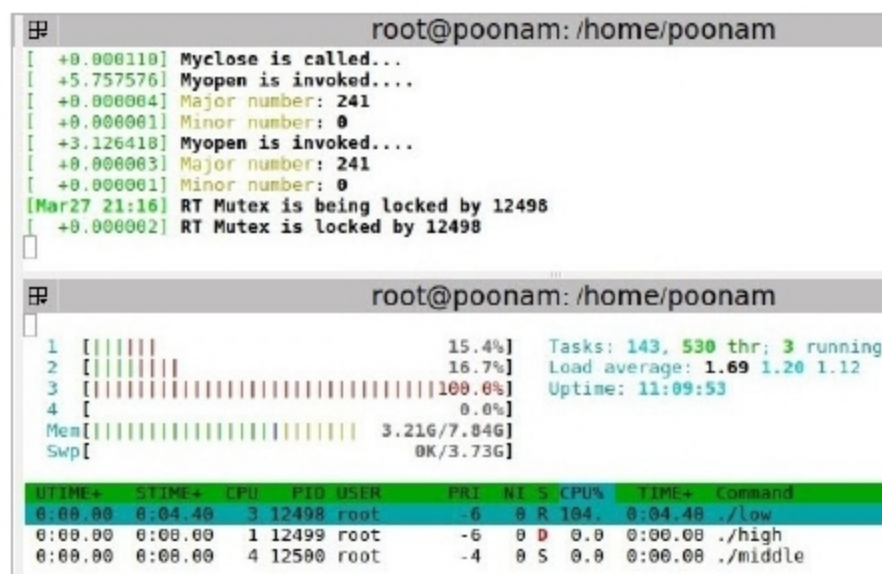


Figure 13: Priority gets inherited for the low priority process

Observation: Here, all processes are running on different processors. So, the middle priority process has no dependency on the other two processes (just like semaphore). For the other two processes, one waits for the other to complete executing the critical section. Also, priority inheritance is observed.

After the experimentation we observed that rt-mutex with priority inheritance can be used to avoid the priority inversion problem at the device driver level. If rt-mutex is used at the device driver level, user programs can be simplified a lot, and the locking and signalling mechanisms can be saved for critical tasks at the user level.

Also, we found that *PRE-EMPTIBLE Kernel* is not a fully pre-emptible kernel. It still doesn't pre-empt a process executing in the critical section. To get a fully pre-emptible

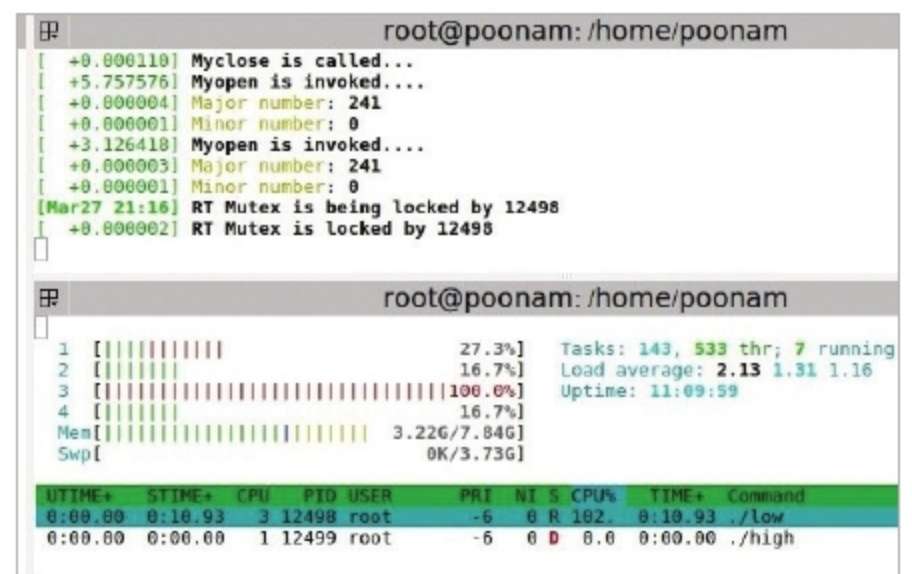


Figure 14: Inherited priority restores after CS

kernel, we need to use an RT pre-emptible patch. **END** 

References

- [1] 'Study of Priority Inversion in Embedded Linux', by Chun-tao Man and Peng Li (IEEE 2006) <https://ieeexplore.ieee.org/document/1692154>
- [2] 'Kernel Corner: A Simple Read and Write Pseudo Character Device Driver' by Dr. B. Thangaraju, *Linux For You*, June 2003, pg.71-73.
- [3] 'Linux Device Drivers' by Jonathan Corbet, pg. 42-70. <https://bootlin.com/doc/books/ldd3.pdf>

 By: Poonam S. Warade and Prof. B. Thangaraju

The authors are associated with the Open Source Technology Lab in the International Institute of Information Technology, Bengaluru. Poonam Warade can be reached at waradepoonam@gmail.com.